

Rezervační systém autopůjčovny

Semestrální projekt TNPW2

Členové týmu a rozdělení odpovědnosti

Student	Business entita	Infrastrukturní role
Veselský Jan (I2400593)	Vehicle - stavový automat vozidla	IR01(State Management), IR02(Dispatcher), IR03(Async), IR04(Router)
Málek Jan (I2400577)	Reservation - stavový automat rezervace	IR05(Selectors), IR06(Views), IR07(Handlers), IR08(Auth)

Architektura projektu

```
src/
├─ entities/                # Business entity se stavovými automaty
│   └─ Vehicle.js          # DRAFT → AVAILABLE →
RENTED/MAINTENANCE/DECOMMISSIONED
│   └─ Reservation.js      # NEW → CONFIRMED → ACTIVE → COMPLETED/CANCELED
├─ infrastructure/         # Infrastrukturní role
│   └─ state.js            # IR01 - globální state management
│   └─ dispatcher.js       # IR02 - centrální dispatch akcí
│   └─ router.js           # IR04 - URL routing a navigace
│   └─ handlers.js         # IR07 - mapování UI na akce
├─ api/                    # Externí komunikace
│   └─ mockApi.js          # IR03 - Mock API s asynchronními operacemi
├─ selectors/              # IR05 - Výběr a transformace dat
│   └─ index.js
├─ views/                  # IR06 - Renderovací logika
│   └─ components.js       # UI komponenty (el, button, input...)
│   └─ vehicles.js         # Pohledy na vozidla
│   └─ reservations.js     # Pohledy na rezervace
│   └─ modals.js           # Modální okna a formuláře
│   └─ layout.js           # Navigace, notifikace, footer
│   └─ auth.js             # IR08 - Autentizace
└─ utils/
```

— main.js	# Vstupní bod aplikace
— dom.js	# DOM pomocné funkce (legacy)

Business entity

Vehicle (Vozidlo)

Stavy: DRAFT → AVAILABLE → RENTED / MAINTENANCE / DECOMMISSIONED

Invarianty:

- Vozidlo ve stavu RENTED nelze smazat ani vyřadit
- Pokud status ≠ AVAILABLE , nelze vytvářet nové CONFIRMED rezervace
- Stav tachometru při návratu nesmí být nižší než při výdeji

Přechody:

- DRAFT → AVAILABLE (admin, validní technické údaje)
- AVAILABLE → RENTED (při aktivaci rezervace)
- AVAILABLE → MAINTENANCE (admin, nahlášena závada)
- AVAILABLE → DECOMMISSIONED (admin, prodej vozu)
- RENTED → AVAILABLE (vrácení vozu v pořádku)
- RENTED → MAINTENANCE (vrácení s poškozením)
- MAINTENANCE → AVAILABLE (servis dokončen)
- MAINTENANCE → DECOMMISSIONED (oprava nerentabilní)

Reservation (Rezervace)

Stavy: NEW → CONFIRMED → ACTIVE → COMPLETED / CANCELED

Invarianty:

- Zákazník může mít v daném čase maximálně jednu ACTIVE rezervaci
- Pokud vehicle.status ≠ AVAILABLE , nelze přejít do CONFIRMED
- Datum vrácení musí být striktně po datu půjčení

Přechody:

- NEW → CONFIRMED (systém/pracovník, vehicle.status = AVAILABLE, platba OK)
- NEW → CANCELED (zákazník, nebo vypršení času)
- CONFIRMED → ACTIVE (pracovník, fyzické předání vozu)

- `CONFIRMED` → `CANCELED` (storno zákazníkem, nedostavení se)
- `ACTIVE` → `COMPLETED` (pracovník, převzetí vozu zpět)
- `ACTIVE` → `CONFIRMED` (chyba obsluhy, návrat stavu)

Infrastrukturní role

IR01 – State Management (Veselský Jan)

- Návrh globálního stavu aplikace (vehicles, reservations, ui, auth)
- Inicializace stavu
- Oddělení doménových a technických dat
- Řízené aktualizace stavu přes definované mutace

IR02 – Dispatcher (Veselský Jan)

- Centrální zpracování akcí
- Interpretace `action.type`
- Volání business funkcí
- Vyvolání změn stavu

IR03 – Asynchronní operace (Veselský Jan)

- Komunikace s Mock API / HTTP API
- Práce s časem a čekáním
- Zpracování SUCCESS / REJECTED / ERROR
- Přechody do loading a error stavů

IR04 – Router (Veselský Jan)

- Mapování URL na aplikační kontext
- Synchronizace adresy prohlížeče se stavem aplikace
- Převod URL → akce
- Reakci na změny historie

IR05 – Selektory (Málek Jan)

- Filtrování kolekcí
- Odvozené hodnoty (`isAvailable` , `canReserve` , `totalPrice`)
- Pojmenování významových stavů aplikace

- Příprava dat a capabilities pro konkrétní pohled

IR06 – Renderovací logika (Málek Jan)

- Převod view-state na UI strukturu
- Podmíněné zobrazení částí UI
- Sestavení DOM stromu

IR07 – Handlers (Málek Jan)

- Definice handlerů (`onReserve` , `onCancel`)
- Mapování interakcí na `dispatch(action)`
- Izolace UI od business logiky

IR08 – Autentizace (Málek Jan)

- Uložení identity uživatele
- Práce s tokenem
- Inicializace autentizačního stavu
- Předávání identity API vrstvě

Kontrola kvality (Code Review)

- Žádná business logika ve View
- Žádná přímá mutace stavu mimo dispatcher
- Žádná autorizace v UI

Spuštění projektu

Otevřete `index.html` v moderním prohlížeči nebo použijte lokální server:

```
# Pomocí Python
python -m http.server 8080
```

```
# Pomocí Node.js (npx serve)
npx serve .
```

```
# Pomocí PHP
php -S localhost:8080
```

Pak otevřete `http://localhost:8080`.

Demo účty

Role	Email	Heslo
Admin	admin@autopujcovna.cz	admin123
Zaměstnanec	zamestnanec@autopujcovna.cz	emp123

Technologie

- Vanilla JavaScript (ES6+ moduly)
- Tailwind CSS (via CDN)
- Žádné externí frameworky (React, Vue, Angular...)
- Custom DOM rendering

Funkcionalita

- Správa vozidel (CRUD, stavový automat, správa tachometru)
- Správa rezervací (vytváření, potvrzení, vydání, vrácení, storno)
- Dashboard s přehledem statistik
- Filtrace a vyhledávání
- Notifikace o změnách
- Responzivní design

Dokumentace akcí a dispatch mechanismu

Architektura SPA – Rezervační systém autopůjčovny

1. STRUKTURA AKCE

Každá akce v systému má následující strukturu:

```
interface Action {  
  type: string;           // Unikátní identifikátor akce
```

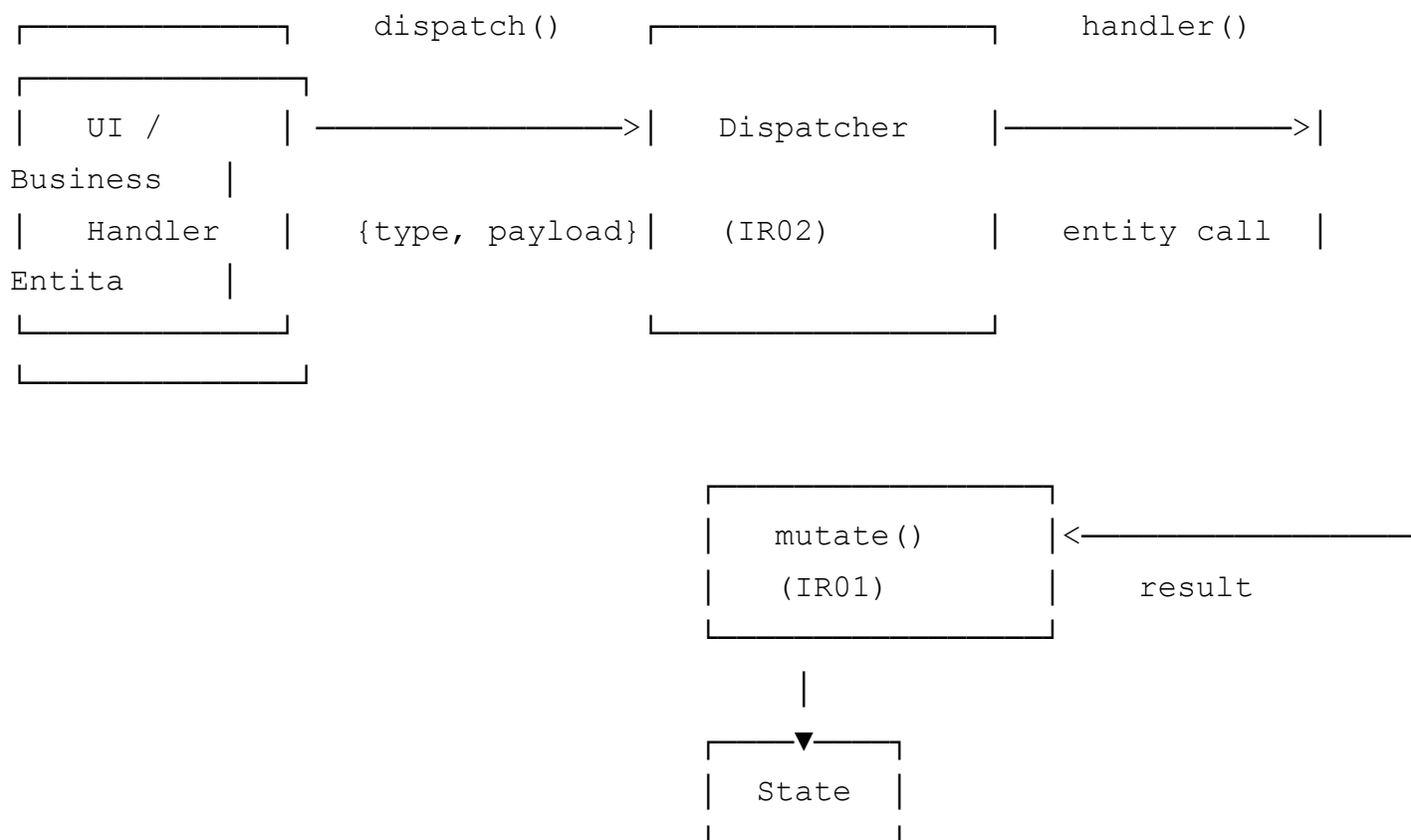
```
payload?: any; // Data předávaná akci (volitelné)
}
```

Příklad:

```
{
  type: 'CREATE_VEHICLE',
  payload: {
    brand: 'Škoda',
    model: 'Octavia',
    year: 2022,
    dailyRate: 800
  }
}
```

2. DISPATCH MECHANISMUS (IR02)

2.1 Architektura



2.2 Flow akce

1. **Vstup** - UI nebo handler zavolá `dispatch(action)`

2. **Validate** - Dispatcher ověří, že akce má platný `type`
3. **Vyhledání handleru** - Pomocí `actionHandlers.get(action.type)`
4. **Volání handleru** - Handler provede:
 - Volání business logiky (Entity metody)
 - Asynchronní operace (IR03)
 - Mutace stavu (IR01)
5. **Vrácení výsledku** - Handler vrátí `{success, data?, error?}`
6. **Notifikace subscriberů** - State notifikuje o změně

2.3 Implementace

```
// src/infrastructure/dispatcher.js

// Registry handlerů
const actionHandlers = new Map();

// Registrace handleru
export function registerAction(type, handler) {
  actionHandlers.set(type, handler);
}

// Centrální dispatch
export async function dispatch(action) {
  if (!action || !action.type) {
    return { success: false, error: 'Invalid action' };
  }

  const handler = actionHandlers.get(action.type);
  if (!handler) {
    return { success: false, error: `No handler for action: ${action.type}` };
  }

  try {
    const result = await handler(action.payload, getState);
    return result;
  } catch (error) {
    return { success: false, error: error.message };
  }
}
```

3. SEZNAM AKCÍ PODLE KATEGORIÍ

3.1 Vehicle akce (IR02 - Veselský Jan)

Akce	Payload	Výsledek	Popis
FETCH_VEHICLES	-	{success, data: Vehicle[]}	Načte všechna vozidla z API
CREATE_VEHICLE	{brand, model, year, licensePlate, mileage, dailyRate, description}	{success, data: Vehicle}	Vytvoří nové vozidlo (status DRAFT)
UPDATE_VEHICLE_STATUS	{vehicleId, newStatus, userRole}	{success, data: Vehicle}	Změní stav vozidla s validací FSM
UPDATE_VEHICLE_MILEAGE	{vehicleId, mileage}	{success, data: Vehicle}	Aktualizuje tachometr (invariant: neklesá)
DELETE_VEHICLE	{vehicleId}	{success}	Smaže vozidlo (invariant: není RENTED)

Příklad volání:

```
const result = await dispatch({
  type: 'UPDATE_VEHICLE_STATUS',
  payload: {
    vehicleId: 'v1',
    newStatus: 'MAINTENANCE',
    userRole: 'admin'
  }
});

// Úspěch: {success: true, data: {id: 'v1', status: 'MAINTENANCE', ...}}
// Neúspěch: {success: false, error: 'Nepovolený přechod: ...'}
```

3.2 Reservation akce (IR02 - Málek Jan)

Akce	Payload	Výsledek	Popis
------	---------	----------	-------

Akce	Payload	Výsledek	Popis
FETCH RESERVATIONS	-	<code>{success, data: Reservation[]}</code>	Načte všechny rezervace z API
CREATE RESERVATION	<code>{vehicleId, startDate, endDate, customerName, customerEmail, notes}</code>	<code>{success, data: Reservation}</code>	Vytvoří novou rezervaci (status NEW)
CONFIRM RESERVATION	<code>{reservationId, userRole}</code>	<code>{success, data: Reservation}</code>	Potvrdí rezervaci (NEW → CONFIRMED)
ACTIVATE RESERVATION	<code>{reservationId, userRole}</code>	<code>{success, data: Reservation}</code>	Vydá vozidlo (CONFIRMED → ACTIVE)
COMPLETE RESERVATION	<code>{reservationId, userRole, finalMileage}</code>	<code>{success, data: Reservation}</code>	Přijme vozidlo (ACTIVE → COMPLETED)
CANCEL RESERVATION	<code>{reservationId, userRole, reason}</code>	<code>{success, data: Reservation}</code>	Zruší rezervaci

Příklad volání s kaskádovými změnami:

```
// ACTIVATE_RESERVATION automaticky mění i stav vozidla
const result = await dispatch({
  type: 'ACTIVATE_RESERVATION',
  payload: {
    reservationId: 'r1',
    userRole: 'admin'
  }
});

// Výsledek:
// - Rezervace: CONFIRMED → ACTIVE
// - Vozidlo: AVAILABLE → RENTED
// - Notifikace: "Vozidlo bylo vydáno klientovi"
```

3.3 UI akce (IR02 - oba)

Akce	Payload	Výsledek	Popis
NAVIGATE	<code>{view, vehicleId?, reservationId?}</code>	<code>{success}</code>	Změní aktuální pohled

Akce	Payload	Výsledek	Popis
OPEN_MODAL	<code>{type, data?}</code>	<code>{success}</code>	Otevře modální okno
CLOSE_MODAL	-	<code>{success}</code>	Zavře modální okno
SET_FILTERS	<code>{type: 'vehicles' 'reservations', filters}</code>	<code>{success}</code>	Nastaví filtry pro seznam
REMOVE_NOTIFICATION	<code>notificationId</code>	<code>{success}</code>	Odstraní notifikaci

3.4 Auth akce (IR08 – Málek Jan)

Akce	Payload	Výsledek	Popis
LOGIN	<code>{email, password}</code>	<code>{success, data: {user, token}}</code>	Přihlásí uživatele
LOGOUT	-	<code>{success}</code>	Odhlásí uživatele

4. MUTACE STAVU (IR01)

Mutace jsou jediný způsob, jak měnit stav aplikace. Volají se z dispatcheru.

4.1 Seznam mutací

Vehicle mutace

- `SET_VEHICLES` - Nahradí celý seznam vozidel
- `ADD_VEHICLE` - Přidá jedno vozidlo
- `UPDATE_VEHICLE` - Aktualizuje existující vozidlo
- `REMOVE_VEHICLE` - Odstraní vozidlo
- `SET_VEHICLES_LOADING` - Nastaví loading flag
- `SET_VEHICLES_ERROR` - Uloží chybu

Reservation mutace

- `SET_RESERVATIONS` - Nahradí celý seznam rezervací
- `ADD_RESERVATION` - Přidá jednu rezervaci

- `UPDATE_RESERVATION` - Aktualizuje existující rezervaci
- `REMOVE_RESERVATION` - Odstraní rezervaci
- `SET_RESERVATIONS_LOADING` - Nastaví loading flag
- `SET_RESERVATIONS_ERROR` - Uloží chybu

UI mutace

- `SET_CURRENT_VIEW` - Změní aktuální pohled
- `SELECT_VEHICLE` - Vybere vozidlo pro detail
- `SELECT_RESERVATION` - Vybere rezervaci pro detail
- `OPEN_MODAL` / `CLOSE_MODAL` - Ovládání modálu
- `ADD_NOTIFICATION` / `REMOVE_NOTIFICATION` - Notifikace
- `SET_VEHICLE_FILTER` / `SET_RESERVATION_FILTER` - Filtry

Auth mutace

- `SET_USER` - Nastaví přihlášeného uživatele
- `SET_TOKEN` - Uloží auth token
- `SET_AUTH_LOADING` - Nastaví loading flag
- `SET_AUTH_ERROR` - Uloží chybu přihlášení
- `LOGOUT` - Vyresetuje auth stav
- `RESET_STATE` - Vyresetuje celý stav aplikace

4.2 Implementace mutace

```
// src/infrastructure/state.js

export function mutate(mutation) {
  const oldState = getState();

  switch (mutation.type) {
    case 'ADD_VEHICLE':
      const vehicleData = mutation.payload;
      currentState.vehicles.byId[vehicleData.id] = vehicleData;
      currentState.vehicles.allIds.push(vehicleData.id);
      break;

    case 'UPDATE_VEHICLE_STATUS':
      const update = mutation.payload;
      if (currentState.vehicles.byId[update.id]) {
```

```

        currentState.vehicles.byId[update.id] = {
            ...currentState.vehicles.byId[update.id],
            ...update,
            updatedAt: new Date().toISOString()
        };
    }
    break;

    // ... další mutace
}

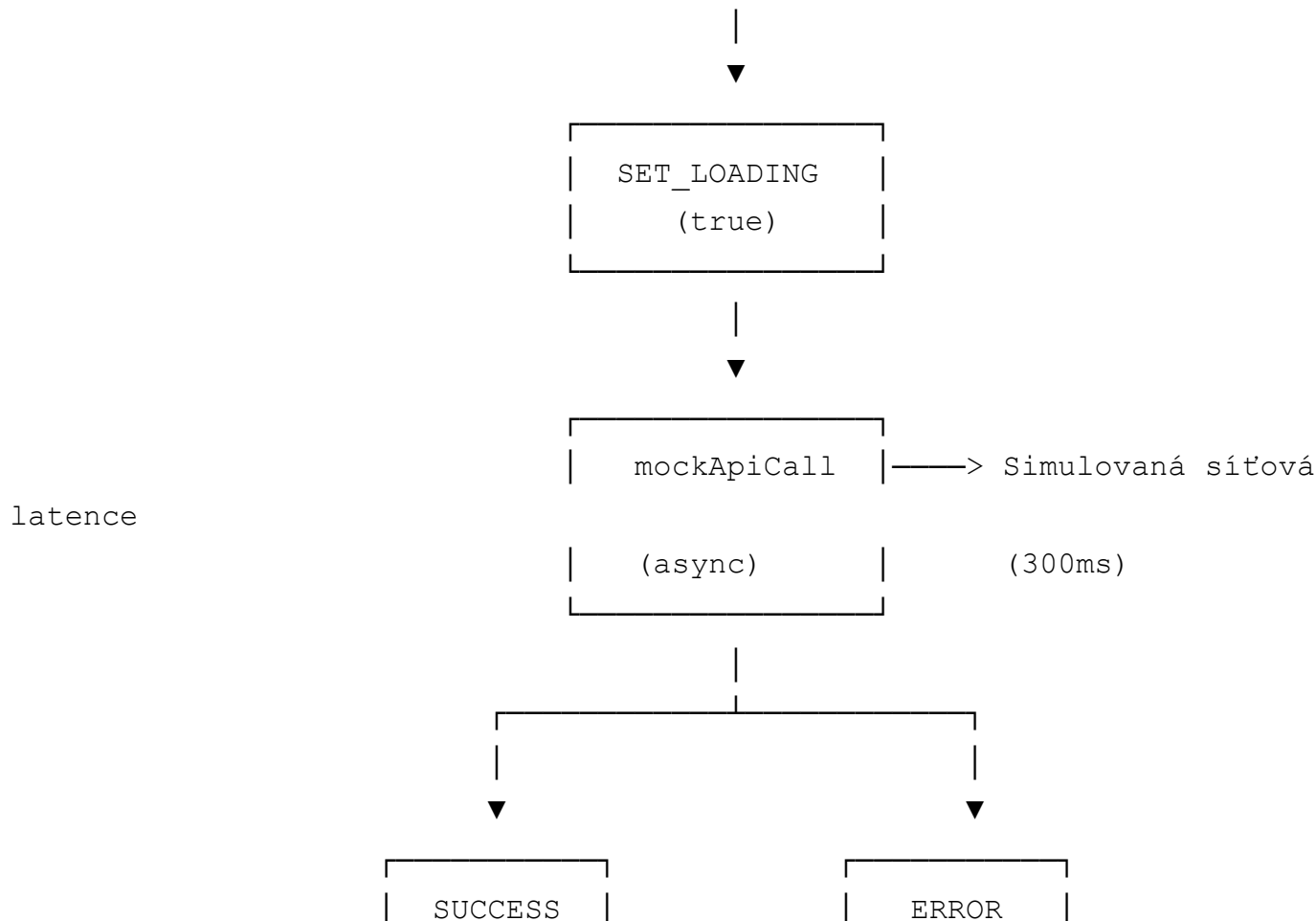
// Notifikace subscriberů o změně
notifySubscribers(oldState, getState(), mutation);
return true;
}

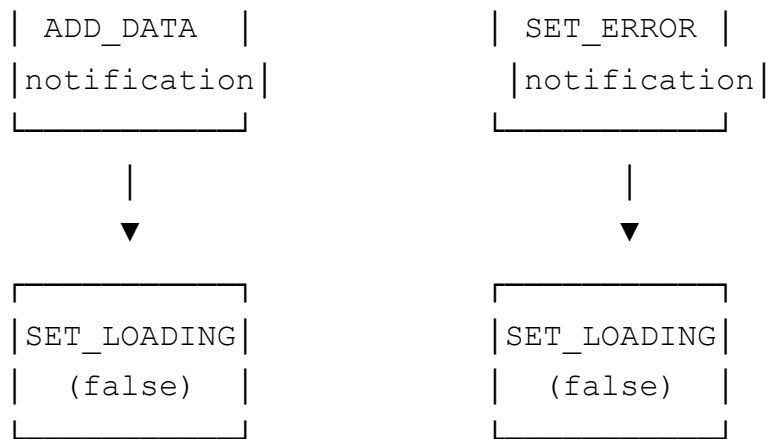
```

5. ASYNCHRONNÍ OPERACE (IR03)

5.1 Flow async akce

UI → dispatch(CREATE_VEHICLE) → handler





5.2 Příklad implementace

```
// src/infrastructure/dispatcher.js

registerAction('FETCH_VEHICLES', async (payload, getState) => {
  // 1. Nastavit loading
  mutate({ type: 'SET_VEHICLES_LOADING', payload: true });

  try {
    // 2. Async volání API (IR03)
    const vehicles = await mockApiCall('GET', '/vehicles');

    // 3. Úspěch - uložit data
    mutate({ type: 'SET_VEHICLES', payload: vehicles });
    mutate({ type: 'SET_VEHICLES_LOADING', payload: false });

    return { success: true, data: vehicles };
  } catch (error) {
    // 4. Chyba - uložit error
    mutate({ type: 'SET_VEHICLES_ERROR', payload: error.message });
    mutate({ type: 'SET_VEHICLES_LOADING', payload: false });

    return { success: false, error: error.message };
  }
});
```

6. ROUTING A AKCE (IR04)

6.1 Mapování URL na akce

URL	Akce	Payload
#/ nebo #/vehicles	NAVIGATE	{view: 'vehicles'}
#/vehicles/:id	NAVIGATE	{view: 'vehicle-detail', vehicleId: ':id'}
#/reservations	NAVIGATE	{view: 'reservations'}
#/reservations/:id	NAVIGATE	{view: 'reservation-detail', reservationId: ':id'}

6.2 Synchronizace stavu → URL

```
// src/infrastructure/router.js
```

```
export function updateUrlFromState(state) {
  const { currentView, selectedVehicleId, selectedReservationId } = state

  let newPath = '/';
  switch (currentView) {
    case 'vehicles': newPath = '/vehicles'; break;
    case 'vehicle-detail':
      newPath = selectedVehicleId ? `/vehicles/${selectedVehicleId}` : '/';
      break;
    case 'reservations': newPath = '/reservations'; break;
    case 'reservation-detail':
      newPath = selectedReservationId ? `/reservations/${selectedReservationId}` : '/';
      break;
  }

  window.history.pushState(null, '', `#${newPath}`);
}
```

7. INVARIANTY A VALIDACE

7.1 Kontroly v dispatchi

```
registerAction('UPDATE_VEHICLE_STATUS', async (payload, getState) => {
  const { vehicleId, newStatus, userRole } = payload;
  const state = getState();
```

```

const vehicleData = state.vehicles.byId[vehicleId];

// 1. Kontrola existence
if (!vehicleData) {
  return { success: false, error: 'Vozidlo nenalezeno' };
}

// 2. Business validace (FSM)
const vehicle = Vehicle.fromJSON(vehicleData);
const result = vehicle.updateStatus(newStatus, userRole);

if (!result.success) {
  // Notifikace o chybě
  mutate({ type: 'ADD_NOTIFICATION', payload: { type: 'error', message:
  return result;
}

// 3. API volání a aktualizace stavu
// ...
});

```

7.2 Seznam kontrolovaných invariantů

Invariant	Kde je kontrolován	Akce
Vozidlo RENTED nelze smazat	<code>Vehicle.canDeleteOrDecommission()</code>	DELETE_VEHICLE
Tachometr nesmí klesat	<code>Vehicle.updateMileage()</code>	UPDATE_VEHICLE_MILEAGE
Přechod musí být v FSM	<code>Vehicle.canTransitionTo()</code>	UPDATE_VEHICLE_STATUS
Datum vrácení > půjčení	<code>Reservation.createReservation()</code>	CREATE_RESERVATION
Max 1 ACTIVE rezervace	<code>selectActiveReservationForCustomer()</code>	CREATE_RESERVATION
Vozidlo AVAILABLE pro CONFIRMED	<code>Reservation.confirm()</code>	CONFIRM_RESERVATION

8. SEZNAM VŠECH AKCÍ (abecedně)

1. `ACTIVATE_RESERVATION` - IR02 (Málek)
2. `CANCEL_RESERVATION` - IR02 (Málek)

3. `CLOSE_MODAL` - IR02
 4. `COMPLETE_RESERVATION` - IR02 (Málek)
 5. `CONFIRM_RESERVATION` - IR02 (Málek)
 6. `CREATE_RESERVATION` - IR02 (Málek)
 7. `CREATE_VEHICLE` - IR02 (Veselský)
 8. `DELETE_VEHICLE` - IR02 (Veselský)
 9. `FETCH_RESERVATIONS` - IR02 (Málek)
 10. `FETCH_VEHICLES` - IR02 (Veselský)
 11. `LOGIN` - IR08 (Málek)
 12. `LOGOUT` - IR08 (Málek)
 13. `NAVIGATE` - IR02
 14. `OPEN_MODAL` - IR02
 15. `REMOVE_NOTIFICATION` - IR02
 16. `SET_FILTERS` - IR02
 17. `UPDATE_RESERVATION` - IR02 (Málek) - interní
 18. `UPDATE_VEHICLE_MILEAGE` - IR02 (Veselský)
 19. `UPDATE_VEHICLE_STATUS` - IR02 (Veselský)
-

9. TESTOVACÍ SCÉNÁŘE PRO DISPATCH

TC-DISP1: Úspěšný průchod akce

Vstup: `dispatch({type: 'FETCH_VEHICLES'})`

Průběh:

1. Dispatcher najde handler
2. Handler nastaví `loading=true`
3. API vrátí data
4. Handler uloží data
5. Handler nastaví `loading=false`
6. Výsledek: `{success: true, data: [...]}`

TC-DISP2: Neúspěšná akce s business chybou

Vstup: `dispatch({type: 'UPDATE_VEHICLE_STATUS', payload: {vehicleId: 'v1', newStatus: 'INVALID', userRole: 'admin'}})`

Průběh:

1. Dispatcher najde handler
2. Handler volá `vehicle.updateStatus()`
3. Business logika vrátí chybu
4. Handler přidá notifikaci
5. Výsledek: `{success: false, error: 'Nepovolený přechod...'}`

TC-DISP3: Neexistující akce

Vstup: `dispatch({type: 'UNKNOWN_ACTION'})`

Výsledek: `{success: false, error: 'No handler for action: UNKNOWN_ACTION'}`

Dokumentace vytvořena dle požadavků zadání bod 8.